

Graded Assignment: An Interpreter for the *Simple* Virtual Machine

The *simple* virtual machine executes programs written in a simple, assembly-type language (specified below). You must write, in a source file called `simple.c`, an implementation of the *simple* virtual machine. The virtual machine must run the program given as the first command-line argument.

General Syntactic Structure of *Simple* Programs

The following is an example of a program that outputs the sum of all the values in its input:

```
# This is a _simple_ program
a = 0
loop:
    # read an input value and store it in register x
    input x e
    # e is TRUE when the input reaches its end
    if e end
    a = a + x
    goto loop
end:
    output a
    halt
```

A *simple* program consists of a sequence of instructions. Instructions can be labeled, so that the execution can jump to those instructions from any point in the program. In the program source, each instruction or label is in its own line of code, so individual instructions and labels can be read line-by-line. The maximum length of a source-code line is 1024 characters. A line may also contain a comment, as illustrated in the example above. Each line defines an instruction as a sequence of space-separated *tokens*. The space and tabulation characters (' ' and '\t') serve as separators. Zero or more space characters may appear at the beginning and at the end of each line. One or more separator characters appear between tokens.

Data Model, Registers, and Memory

The *simple* virtual machine operates on `int` data. Registers, memory cells, input and output values, and computed values are all of type `int` as defined by your C compiler. For instructions that compute or check logical conditions, `int` values are interpreted as Boolean values in the same way as in C. That is, 0 is *false*, and anything other than 0 is *true*.

The machine has 26 registers named *a*, *b*, *c*, ..., *z*. Thus a single letter *a*, *b*, ..., or *z* in the source code is a reference to the corresponding register. The machine also has a memory that can be addressed using any register. In other words, the memory consists of an array (of `int` objects) that can be indexed by a register value. The base address is 0, and the memory array virtually extends to all the memory cells addressable by a non-negative `int` value. In the source code, a memory reference is written with the character `@` immediately followed by a register name. For example, `@i` refers to the memory cell whose address is the value of register *i*.

We say that the memory extends “virtually” up to all the addressable cells in the sense that the virtual machine must allocate memory *as needed* by the program. In practice, with a program like the one above that does not use the memory at all (beyond registers), the virtual machine should allocate no memory at all. However, whenever the program stores a value in memory, the virtual machine must make sure that the requested memory cell is available.

Input/Output

The *simple* virtual machine has one input channel and one output channel. The standard input and standard output are used as input and output channels, with a decimal textual encoding of integer values. In practice, this the input channel must consist of decimal numbers separated by space characters (including but not necessarily new-line characters), and therefore can be read by `scanf("%d", ...)`. The output channel must consist of decimal numbers each terminated by a new-line character, and therefore can be written by `printf("%d\n", ...)`.

In the source code, the *input* instruction is defined by three, tokens:

`input refv refe`

ref_v is a reference (register or memory) where the instruction stores the *value* read from the input. *ref_e* is a reference in which the instruction stores a possible *end-of-file or error* condition as a Boolean value. For example `input x e` attempts to read an integer from the input channel, and when successful stores its value in register *x* and a *false* value in register *e* indicating that there are no error or end-of-file conditions. In case of error or end-of-input, the value stored in *v* is indeterminate, and the value of register *e* is *true*.

The *output* instruction consists of two tokens:

`output value`

The *value* token is a reference or a literal value, and it is what is printed onto the output channel.

Labels and Program Flow

A program consists of a sequence of instructions. Some instructions may be labeled. In the source code, a label is defined by a single token consisting of a non-empty string of at most 50 characters followed immediately by a colon character (`:`), without any space in between. A label applies to the instruction that immediately *follows* the label in the source code. A program where a label is followed by another label, or where there is no instruction following

a label is invalid.

The execution of a program proceeds from one instruction to the next, except for the *goto*, *if*, and *halt* instructions detailed below. The execution may also be interrupted due to a runtime error. These cases are also detailed below.

The *goto* instruction is defined by two tokens:

goto label

The execution proceeds with the instruction labeled *label*. If the given label does not exist, the program is invalid.

The *if* instruction is defined by three tokens:

if cond label

If the value of *cond*, which is a reference to a register or memory cell, is *true*, then the execution continues with the instruction labeled *label*. Otherwise, the execution continues with the next instruction.

The *halt* instruction is defined by a single token:

halt

and causes the program to terminate immediately.

A program also terminates after the last instruction, if that is not a *goto* instruction.

Assignments and Expressions

An *assignment* instruction is defined by three tokens:

dest = src

dest is a reference to a register or memory cell; *src* is a literal value or a reference. The object referenced by *dest* takes the value of *src*.

An assignment can also take a value computed from a binary operator:

dest = a op b

Here, too, *dest* is a reference to the object whose value will be assigned. The expression on the right-hand side of the *=* token consists of three tokens: *a* and *b* are literal values or references; *op* is one of the following tokens:

+

addition. Computes and stores the value of the addition $a + b$ in the destination register or memory cell.

-

subtraction. Computes and stores the value of the difference $a - b$ into the destination register or memory cell.

*

multiplication. Computes and stores the value of the multiplication $a \cdot b$ into the

	destination register or memory cell.
/	integer division Computes and stores the value of the integer division a/b . If the b is zero, the program terminates immediately with an error.
==	equals comparison. Computes and stores the Boolean value $a = b$ into the destination register or memory cell.
!=	not-equals comparison. Computes and stores the Boolean value $a \neq b$ into the destination register or memory cell.
<	less-than comparison. Computes and stores the Boolean value $a < b$ into the destination register or memory cell.
>	greater-than comparison. Computes and stores the Boolean value $a > b$ into the destination register or memory cell.
<=	less-than or equals comparison. Computes and stores the Boolean value $a \leq b$ into the destination register or memory cell.
>=	greater-than or equals comparison. Computes and stores the Boolean value $a \geq b$ into the destination register or memory cell.

Program Validity and Dynamic Failures

If the given program is *invalid*, then the the virtual machine must return immediately with an error and a failure exit status. A program is invalid if the syntax of the program does not conform with the above specifications.

The execution might also terminate, with an appropriate error message and exist status code, when the execution of an instruction fails, or due to error conditions within the execution environment. Such errors include:

- access to an invalid memory address (e.g., a negative address)
- memory allocation failure
- input/output failures
- division by zero

Example Program

The following is an implementation of the *insertion-sort* algorithm.

```

    n = 0
loop:
    input x e
    if e sort
    @n = x
    n = n + 1
    goto loop

```

```
sort:
    i = 1
next_i:
    c = i == n
    if c print
    j = i
    next_j:
        k = j - 1
        c = @j >= @k
        if c end_j
        t = @j
        @j = @k
        @k = t
        j = j - 1
        c = j == 0
        if c end_j
        goto next_j
    end_j:
        i = i + 1
        goto next_i
print:
    i = 0
print_next:
    c = i == n
    if c end
    output @i
    i = i + 1
    goto print_next
end:
    halt
```